

Esame Programmazione di Sistema / API Programming

Data: 2025-06-17

QUESTIONS

Theory: Question 1

Si considerino le seguenti strutture dati e rispettive porzioni di codice. Per ciascuna di esse si indichi la dimensione di memoria allocata nello stack e nello heap, ipotizzando un'architettura a 64 bit.

```
1 let boxed: Box<u32> = vec![10, 20, 30, 40, 50].into_boxed_slice();
2 let slice_ref: &[u32] = &boxed[1..4];
```

Domande:

1. Qual è la dimensione della variabile `boxed` nello stack?
2. Quanta memoria è allocata nello heap dopo l'esecuzione del codice?
3. Qual è lo spazio di memoria occupato da `slice_ref` nello stack?
4. La slice `slice_ref` punta alla stessa memoria heap della `boxed`?

Si considerino le seguenti strutture dati e porzioni di codice. Per ciascuna di esse si indichi la dimensione di memoria allocata nello stack e nello heap, ipotizzando un'architettura a 64 bit.

```
1 use std::rc::Rc;
2 use std::cell::RefCell;
3 let shared_vec: Rc<RefCell<Vec<u8>>> = Rc::new(RefCell::new(vec![1, 2, 3]));
4 let shared_clone = Rc::clone(&shared_vec);
5 shared_clone.borrow_mut().push(4);
```

Domande:

1. Quanta memoria è usata nello stack da `shared_vec` e `shared_clone`?
2. Quanto spazio è allocato nello heap in totale (considerando `Rc`, `RefCell`, e `Vec`)?
3. Dopo l'esecuzione del codice, quanti riferimenti forti e deboli esistono?
4. È possibile accedere contemporaneamente in lettura da `shared_vec` e in scrittura da `shared_clone`?

Theory: Question 2

Nel contesto della programmazione generica, si spieghi che cosa si intende per **monomorfizzazione**.

Si scriva una porzione di codice che mostri un esempio pratico in cui la monomorfizzazione viene applicata, spiegando perché tale esempio è rilevante.

Si indichino i principali punti di forza e i punti di debolezza di tale proprietà, facendo un raffronto rispetto agli approcci alla programmazione generica offerti da altri linguaggi di programmazione.

Theory: Question 3

Si consideri il seguente programma Rust che usa due thread per gestire una coda condivisa. Le righe sono numerate.

```
1 use std::sync::{Arc, Mutex, Condvar};
2 use std::thread;
3 use std::collections::VecDeque;
4
5 fn main() {
6     let queue = Arc::new((Mutex::new(VecDeque::new()), Condvar::new()));
7
8     let queue_producer = Arc::clone(&queue);
9     let queue_cons = Arc::clone(&queue);
10
11     let producer = thread::spawn(move || {
12         let (lock, cvar) = &*queue_producer;
13         for i in 1..=3 {
14             thread::sleep(std::time::Duration::from_millis(500));
15             let mut queue = lock.lock().unwrap();
16             queue.push_back(i);
17             cvar.notify_one();
18         }
19     });
20
21     let consumer = thread::spawn(move || {
22         let (lock, cvar) = &*queue_cons;
23         loop {
24             let mut queue = lock.lock().unwrap();
25             while queue.is_empty() {
26                 queue = cvar.wait(queue).unwrap();
27             }
28             if let Some(val) = queue.pop_front() {
29                 println!("Consumer 1: got {}", val);
30             } else {
31                 println!("(A)");
32             }
33         }
34     });
35
36     producer.join().unwrap();
37     consumer.join().unwrap();
38 }
```

Domande:

1. Descrivere il comportamento complessivo del programma e spiegare cosa fanno i 2 thread.
2. È possibile che venga stampato (A) alla riga 31? Se sì, in quali condizioni?
3. Il programma termina? Se no, spiegare perché e proporre una modifica per permettere una terminazione pulita dei thread consumatori. Nella soluzione proposta occorre indicare le (eventuali) istruzioni da cancellare e le (eventuali) istruzioni da aggiungere, specificando tra quali righe deve avvenire l'inserimento.
4. Ci sono potenziali race condition o deadlock? Giustificare la risposta.

Programming: TokenManager

Un applicativo software multithread fa accesso ai servizi di un server remoto, attraverso richieste di tipo HTTP.

Tali richieste devono includere un token di sicurezza che identifica l'applicativo stesso e ne autorizza l'accesso.

Per motivi di sicurezza, il token ha una validità limitata nel tempo (qualche minuto) e deve essere rinnovato alla sua scadenza.

Il token viene ottenuto attraverso una funzione (fornita esternamente e conforme al tipo `TokenAcquirer`) che restituisce alternativamente un token e la sua data di scadenza o un messaggio di errore se non è possibile fornirlo.

Poiché la emissione richiede un tempo apprezzabile (da alcune centinaia di millisecondi ad alcuni secondi), si vuole centralizzare la gestione del token, per evitare che più thread ne facciano richiesta in contemporanea. A tale scopo deve essere implementata la struct `TokenManager` che si occupa di gestire il rilascio, il rinnovo e la messa a disposizione del token a chi ne abbia bisogno, secondo la logica di seguito indicata.

La struct `TokenManager` offre i seguenti metodi:

```
1 type TokenAcquirer = dyn Fn() -> Result<(String, Instant), String> + Sync +  
   Send;  
2  
3 pub fn new(acquire_token: Box<TokenAcquirer>) -> Self  
4 pub fn get_token(&self) -> Result<String, String>  
5 pub fn try_get_token(&self) -> Option<String>
```

Al proprio interno, la struct `TokenManager` mantiene 3 possibili stati:

- `Empty` - indica che non è ancora stato richiesto alcun token
- `Pending` - indica che è in corso una richiesta di acquisizione del token
- `Valid` - indica che è disponibile un token in corso di validità



Il metodo `new(...)` riceve il puntatore alla funzione in grado di acquisire il token. Essa opera in modalità pigra e si limita a creare un'istanza della struttura con le necessarie informazioni per gestire il suo successivo comportamento.

Il metodo `get_token(...)` implementa il seguente comportamento:

- Se lo stato è **Empty**, passa allo stato **Pending** e invoca la funzione per acquisire il token; se questa ritorna un risultato valido, memorizza il token e la sua scadenza, porta lo stato a **Valid** e restituisce copia del token stesso; se, invece, questa restituisce un errore, pone lo stato a **Empty** e restituisce l'errore ricevuto. Se lo stato è **Pending**, attende senza consumare cicli di CPU che questo passi ad un altro valore, dopodiché si comporta di conseguenza.
- Se lo stato è **Valid** e il token non risulta ancora scaduto, ne restituisce una copia; altrimenti pone lo stato ad **Pending** e inizia una richiesta di acquisizione, come indicato sopra.

Il metodo `try_get_token(...)` implementa il seguente comportamento: Se lo stato è **Valid** e il token non è scaduto, restituisce una copia del token opportunamente incapsulata in un oggetto di tipo `Option`. In tutti gli altri casi restituisce `None`.

Si implementi tale struttura nel linguaggio Rust.

A supporto della validazione del codice realizzato si considerino i seguenti test (due dei quali sono forniti con la relativa implementazione, i restanti sono solo indicati e devono essere opportunamente completati):

```

1 #[test]
2 fn a_new_manager_contains_no_token() {
3     let a: Box<TokenAcquirer> = Box::new(|| Err("failure".to_string()));
4     let manager = TokenManager::new(a);
5     assert!(manager.try_get_token().is_none());
6 }
7
8 #[test]
9 fn a_failing_acquirer_always_returns_an_error() {
10    let a: Box<TokenAcquirer> = Box::new(|| Err("failure".to_string()));
11    let manager = TokenManager::new(a);
12    assert_eq!(manager.get_token(), Err("failure".to_string()));
13    assert_eq!(manager.get_token(), Err("failure".to_string()));
14 }
15
16 #[test]
17 fn a_successful_acquirer_always_returns_success() {
18     //... to be implemented
19 }
20
21 #[test]
22 fn a_slow_acquirer_causes_other_threads_to_wait() {
23     //... to be implemented
24 }

```

SOLUTIONS

Theory: Question 1

Parte 1:

1. La variabile `boxed` nello stack è un *fat pointer*. Trattandosi di un puntatore a un dato `unsized (Box<[u32]>)`, contiene l'indirizzo dell'array nello heap (8 byte) e la sua lunghezza (8 byte). Pertanto la dimensione è $8 + 8 = 16$ byte.
2. Nello heap sono allocati i dati della slice, ovvero 5 elementi di tipo `u32`. Poiché ogni `u32` occupa 4 byte, lo spazio allocato nello heap è di $5 \times 4 = 20$ byte.
3. La variabile `slice_ref` è una slice (`&[u32]`), ovvero un altro *fat pointer* formato da puntatore e lunghezza. Occupa anch'essa $8 + 8 = 16$ byte nello stack.
4. Sì, la slice `slice_ref` punta direttamente alla stessa memoria heap gestita da `boxed`. In particolare, punta a partire dal secondo elemento dell'array allocato.

Parte 2:

1. `shared_vec` e `shared_clone` sono smart pointer di tipo `Rc`. Ognuno di essi è un normale puntatore all'allocazione sullo heap. In un'architettura a 64 bit occupano 8 byte ciascuno (per un totale di 16 byte nello stack).
2. L'allocazione complessiva nello heap comprende il blocco gestito da `Rc`, che include i contatori di riferimenti (strong e weak, tipicamente $8+8 = 16$ byte) più il dato `RefCell<Vec<u8>`. Il `RefCell` ha il suo stato di *borrow* (tipicamente `isize`, 8 byte). Il `Vec` occupa 24 byte (puntatore, capacità, lunghezza). Infine, il buffer del `Vec` allocato in un blocco separato o nello stesso calcolo logico, per contenere `u8`. Al momento della sua creazione `vec![1, 2, 3]` alloca (spesso) una capacità di 3 o 4 byte. Considerando i dati stretti e le struct: $16 (\text{Rc}) + 8 (\text{RefCell}) + 24 (\text{Vec}) = 48$ byte della struct esterna, più i byte del buffer (inizialmente 3 byte, o 4 dopo il `push(4)` in un eventuale riallocazione). La somma dei metadati è 48 byte; coi dati veri $48 + \text{capacity} \approx 52$ byte totali.
3. Dopo l'esecuzione del codice (prima che le variabili vadano fuori scope), ci sono **2** riferimenti forti (`shared_vec` e `shared_clone`) e **0** riferimenti deboli.
4. **No**, non è possibile. Rust vieta avere contemporaneamente un prestito mutabile (scrittura) e uno o più prestiti immutabili (lettura). Con `RefCell` la regola è applicata a *runtime*: se si invoca `borrow_mut()` mentre è attivo un `borrow()`, il programma fa *panic*.

Theory: Question 2

Monomorfizzazione:

La monomorfizzazione è la tecnica utilizzata dal compilatore Rust per risolvere il polimorfismo statico. Quando si utilizza una funzione o una struttura generica, il compilatore crea copie indipendenti (*istanze*) del codice per ciascun tipo concreto con cui viene utilizzata durante la compilazione. Invece di usare un dispatch dinamico a runtime, il compilatore "sostituisce" il parametro di tipo generico con il tipo specifico, generando funzioni monomorfiche.

Esempio Pratico:

```
1 fn print_length<T: AsRef<str>>>(item: T) {  
2     println!("Length: {}", item.as_ref().len());  
3 }  
4  
5 fn main() {
```

```

6   let s = String::from("hello");
7   let str_slice: &str = "world";
8
9   // Il compilatore genera due versioni di print_length:
10  // 1. print_length_String(item: String)
11  // 2. print_length_str(item: &str)
12  print_length(s);
13  print_length(str_slice);
14 }

```

Questo esempio è rilevante perché mostra come Rust riesca a invocare il metodo `len()` sui tipi concreti in modo diretto, permettendo inlining e ottimizzazioni.

Punti di forza e debolezza:

- **Punti di forza:** Essendo il dispatch statico, si evita l'overhead della risoluzione dinamica (niente virtual table, nessun puntatore in più a runtime), migliorando notevolmente le performance.
- **Punti di debolezza:** Causa aumento della dimensione dei file binari generati (*code bloat*) e un maggiore tempo di compilazione, poiché il compilatore deve generare e ottimizzare più copie dello stesso codice. In linguaggi come Java, per contro, l'uso di *type erasure* produce un binario più piccolo perché condivide la stessa istanza compilata tra i tipi reference, pagando però un costo in performance.

Theory: Question 3

1. **Comportamento complessivo e dei 2 thread:** Il main thread istanzia una coda condivisa sincronizzata tramite `Mutex` e `Condvar` e lancia due thread. Il **producer** aggiunge alla coda i numeri da 1 a 3 distanziati di 500ms l'uno dall'altro, inviando una notifica `notify_one()` alla `Condvar` dopo ogni aggiunta, poi termina. Il **consumer** esegue un loop infinito: acquisisce il lock, attende passivamente (`wait`) sulla `Condvar` finché la coda è vuota. Quando viene risvegliato e la coda ha elementi, ne estrae uno e lo stampa.
2. **Stampa di (A): No**, non è mai possibile che venga stampato (A). Questo perché l'istruzione precedente `while queue.is_empty() { queue = cvar.wait(queue).unwrap(); }` garantisce che si esca dall'attesa e si prosegua solo se la coda ha almeno un elemento (ovviando al problema delle false wake-up). Pertanto, il controllo `if let Some(val) = queue.pop_front()` andrà sempre a buon fine (entrando nel branch `Some`) e il blocco `else` non sarà mai raggiunto.
3. **Il programma termina? No.** Il thread **producer** termina regolarmente dopo aver aggiunto i 3 elementi. Il thread **consumer**, però, è in un loop infinito; una volta che la coda torna vuota (dopo aver stampato il "3"), aspetterà sulla `Condvar` indefinitamente. Poiché il main esegue `consumer.join().unwrap();`, aspetterà a sua volta all'infinito il **consumer**.
Modifica proposta: per permettere la terminazione occorre cambiare il tipo della coda, usando ad esempio `Option<i32>` o creare un enum `Message { Val(i32), Stop }`.
Modifiche tra riga 18 e 19 (nel producer):

```

1   let mut queue = lock.lock().unwrap();
2   queue.push_back(None); // Segnale di terminazione
3   cvar.notify_one();
4

```

Modifiche tra riga 28 e 32 (nel consumer): Sostituire da riga 28 a 32 con:

```

1   if let Some(val_opt) = queue.pop_front() {
2       if let Some(val) = val_opt {
3           println!("Consumer 1: got {}", val);

```

```

4         } else {
5             break; // Abbiamo ricevuto il segnale None, usciamo dal loop
6         }
7     }
8

```

4. **Race condition o deadlock?** Nessuna **race condition** sui dati: lo stato condiviso è correttamente protetto dal **Mutex** e Rust impone in fase di compilazione l'uso esclusivo del dato (queue). Non c'è alcun **deadlock** circolare, ma c'è una situazione di **starvation/livelock** indotta logicamente: il consumatore aspetta un segnale che non arriverà mai. In senso stretto, Rust non garantisce assenza di deadlock o loop infiniti, che infatti si manifesta qui bloccando il main.

Programming: TokenManager

Di seguito la soluzione per la struct `TokenManager` in Rust. L'acquisizione del token deve avvenire **al di fuori del blocco lockato** del `Mutex` per permettere agli altri thread di osservare lo stato `Pending` ed effettuare una `wait()` sulla `Condvar`.

```

1 use std::sync::{Arc, Condvar, Mutex};
2 use std::time::{Duration, Instant};
3
4 pub type TokenAcquirer = dyn Fn() -> Result<(String, Instant), String> + Sync +
    Send;
5
6 #[derive(Clone, PartialEq)]
7 enum State {
8     Empty,
9     Pending,
10    Valid((String, Instant)),
11 }
12
13 pub struct TokenManager {
14     mutex: Mutex<State>,
15     condvar: Condvar,
16     acquire_token: Box<TokenAcquirer>,
17 }
18
19 impl TokenManager {
20     pub fn new(acquire_token: Box<TokenAcquirer>) -> Self {
21         Self {
22             mutex: Mutex::new(State::Empty),
23             condvar: Condvar::new(),
24             acquire_token,
25         }
26     }
27
28     pub fn get_token(&self) -> Result<String, String> {
29         let mut lock = self.mutex.lock().unwrap();
30
31         loop {
32             match &*lock {
33                 State::Empty => {
34                     // Imposto lo stato a Pending in modo che gli altri thread
35                     attendano
36                     *lock = State::Pending;
37                     // IMPORTANTE: rilascio il lock prima di eseguire l'
38                     acquisizione
39                     drop(lock);
40
41                     let result = (self.acquire_token)();
42

```

```

40
41 // Riacquisisco il lock per aggiornare il nuovo stato
42 lock = self.mutex.lock().unwrap();
43
44 match result {
45     Ok((token, expiration)) => {
46         *lock = State::Valid((token.clone(), expiration));
47         self.condvar.notify_all();
48         return Ok(token);
49     }
50     Err(e) => {
51         *lock = State::Empty;
52         self.condvar.notify_all();
53         return Err(e);
54     }
55 }
56 }
57 State::Pending => {
58     // Se qualcuno sta acquisendo il token, attendo un
59     // cambiamento di stato
60     lock = self.condvar.wait_while(lock, |state| *state ==
61     State::Pending).unwrap();
62     }
63     State::Valid((token, expiration)) => {
64         if *expiration > Instant::now() {
65             return Ok(token.clone());
66         } else {
67             // Se il token e' scaduto, si riporta lo stato a Empty
68             // in modo che la prossima iterazione del loop ne acquisisca
69             // uno nuovo
70             *lock = State::Empty;
71         }
72     }
73 }
74 }
75 }
76
77 pub fn try_get_token(&self) -> Option<String> {
78     let lock = self.mutex.lock().unwrap();
79     if let State::Valid((token, expiration)) = &*lock {
80         if *expiration > Instant::now() {
81             return Some(token.clone());
82         }
83     }
84     None
85 }
86
87 #[cfg(test)]
88 mod tests {
89     use super::*;
90     use std::thread;
91
92     #[test]
93     fn a_new_manager_contains_no_token() {
94         let a: Box<TokenAcquirer> = Box::new(|| Err("failure".to_string()));
95         let manager = TokenManager::new(a);
96         assert!(manager.try_get_token().is_none());
97     }
98
99     #[test]
100     fn a_failing_acquirer_always_returns_an_error() {

```

```

99     let a: Box<TokenAcquirer> = Box::new(|| Err("failure".to_string()));
100     let manager = TokenManager::new(a);
101     assert_eq!(manager.get_token(), Err("failure".to_string()));
102     assert_eq!(manager.get_token(), Err("failure".to_string()));
103 }
104
105 #[test]
106 fn a_successful_acquirer_always_returns_success() {
107     let a: Box<TokenAcquirer> = Box::new(|| Ok("my_token".to_string(),
Instant::now() + Duration::from_secs(60)));
108     let manager = TokenManager::new(a);
109     assert_eq!(manager.get_token(), Ok("my_token".to_string()));
110     assert_eq!(manager.try_get_token(), Some("my_token".to_string()));
111 }
112
113 #[test]
114 fn a_slow_acquirer_causes_other_threads_to_wait() {
115     // Simuliamo un'acquisizione lenta che richiede 300ms
116     let a: Box<TokenAcquirer> = Box::new(|| {
117         thread::sleep(Duration::from_millis(300));
118         Ok("slow_token".to_string(), Instant::now() + Duration::from_secs
(60)))
119     });
120
121     let manager = Arc::new(TokenManager::new(a));
122     let m1 = Arc::clone(&manager);
123     let m2 = Arc::clone(&manager);
124
125     let t1 = thread::spawn(move || {
126         m1.get_token()
127     });
128
129     let t2 = thread::spawn(move || {
130         // Diamo tempo a t1 di iniziare l'acquisizione (mettere il manager
in Pending)
131         thread::sleep(Duration::from_millis(50));
132         m2.get_token()
133     });
134
135     let res1 = t1.join().unwrap();
136     let res2 = t2.join().unwrap();
137
138     assert_eq!(res1, Ok("slow_token".to_string()));
139     assert_eq!(res2, Ok("slow_token".to_string()));
140 }
141 }

```